Technology Stack

Prototype as built, and the production target

Layer	Prototype	Production	Why
Runtime and API	Python 3.10+, command line	Python, FastAPI, job queue for large sets	One language end to end
Vector PDF parsing	PyMuPDF	PyMuPDF (commercial licence) + OCR service for scanned sheets	Exact text coordinates and line work from CAD exports
Native model data	Revit schedule export reader	+ IfcOpenShell, Revit API / pyRevit exports	The model is the source of truth for authored projects
Schedules and tables	PyMuPDF ruled-table detection	Same, plus a vendor fallback	Every cell has a citable location
Specifications	CSI section and paragraph parser	Same	Paragraph-level citations
Bill of quantities	openpyxl	openpyxl	Exact cell addresses
Units and arithmetic	Exact-fraction imperial and metric parser	Same	No rounding drift in dimension sums
Data model	Pydantic	Pydantic	Typed findings with provenance
Revision overlay	OpenCV, NumPy	Same	Visual aid alongside the semantic change list
LLM (optional)	Claude Opus 5.5, structured output	Pinned current Claude model with zero data retention; local open-weight model for strict-NDA work	Structured output and verbatim quotes for the evidence gate
Evidence gate	Re-verification with PyMuPDF and openpyxl	Same	No unverified citation is shown
Review interface	Streamlit	React + PDF.js / APS viewer, single sign-on	Multi-user review with audit trail
Automation	Folder watcher + n8n workflows	Same, plus ACC webhooks	Runs when a new revision is uploaded
Issue tracking	ACC Issues API (accepted findings only)	Same	Findings land where the team already works
Storage	JSON / JSONL files	PostgreSQL	Findings, decisions and run history with access control
Testing and evaluation	unittest, synthetic benchmark, evaluation harness	+ CI regression on every rule or model change	Measured accuracy, no silent regressions